

Dexamine: A Python Package to Examine Decentralized Exchange Data

26 January 2024

Summary

Dexamine is a Python package designed to streamline and facilitate the parsing of decentralized exchange (DEX) data on the Ethereum blockchain. Developed in response to the growing complexity and volume of DEX transactions, this tool provides researchers, developers, and analysts with a means to efficiently parse and access Ethereum-based DEX data. **Dexamine** version 0.1.0 supports Uniswap V2 and Uniswap V3 on Ethereum mainnet.

The package leverages fundamental Ethereum concepts, such as blocks, transactions, and state transitions, outlined in the Ethereum white paper (Buterin 2013) and the Ethereum yellow paper (Wood 2014), as well as documentation and papers for specific DEXes such as Uniswap V2 (Adams, Zinsmeister, and Robinson 2020) and Uniswap V3 (Adams et al. 2021).

By integrating functionalities compatible with Ethereum client implementations like **Erigon** (Erigon Team 2022), and drawing on the data indexing capabilities akin to **Trueblocks** (TrueBlocks Team 2022), **Dexamine** offers a comprehensive solution to parse all trade data from a DEX.

Statement of need

The Ethereum blockchain, since its inception, has evolved into a complex ecosystem, especially with the rise of DeFi applications. With this evolution comes the challenge of effectively parsing and analyzing the vast and intricate data generated by DeFi protocols. Existing solutions like **Erigon** (Erigon Team 2022) provide optimized Ethereum client implementations, while tools like **Trueblocks** (TrueBlocks Team 2022) offer efficient blockchain data indexing. Nevertheless, there remains a gap in the form of a dedicated tool specifically designed for parsing DEX data within the Ethereum ecosystem.

The **Dexamine** package addresses this gap by providing a solution that simplifies the complexities involved in parsing DEX transaction data. This is important

such that researchers, developers, and analysts do not have to rely on third-party data. **Dexamine** also unlocks deeper analysis of DEX events as all meta-data surrounding the transactions are recorded, these data are not present in typical trade data sets.

It is not only a response to the technical needs identified in the DeFi space but also an embodiment of the open-source ethos that Ethereum was built upon.

Research project utilizing “Dexamine” Mention (if applicable) a representative set of past or ongoing research projects using the software and recent scholarly publications enabled by it.

Dexamine parsing

The data parsed from a DEX event (**swap**, **mint**, and **burn**) by **Dexamine** comes from two sources. First, the DEX event parameters, such as the traded amounts in a **swap**, are parsed from the transaction logs produced by the EVM (Ethereum Virtual Machine). Second, all metadata, such as **block** and **gas**, associated with the transactions are collected.

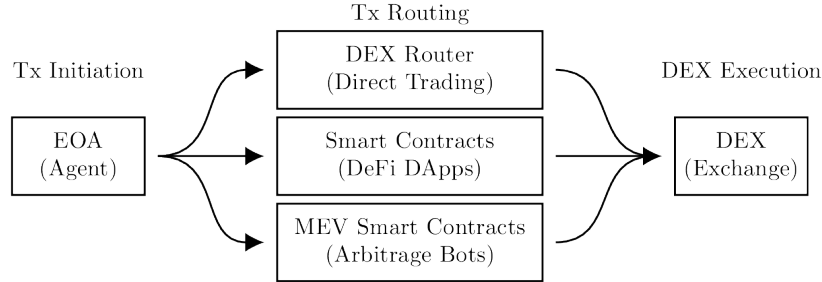


Figure 1: DEX Route Classification.

Additionally, **Dexamine** identifies the route which each transaction takes before execution at the DEX. Figure Figure 1 shows the three ways a transaction can be routed before arriving at a DEX. The route is classified using the **to_address** from the transaction data, where it is match to the DEX router contracts as well as a known list of MEV accounts by *Etherscan*.¹ This allows researchers to carefully study the market impact and trading of three important transaction types.

¹<https://etherscan.io/accounts/label/mev-bot>

Dexamine example workflow

Figure 2 shows an example workflow of how **Dexamine** can be used locally together with an Ethereum archive node and a blockchain indexer. **Dexamine** is agnostic to which Ethereum archive node software is used and what indexer is used, as long as the input data to **Dexamine** adheres to the correct format.

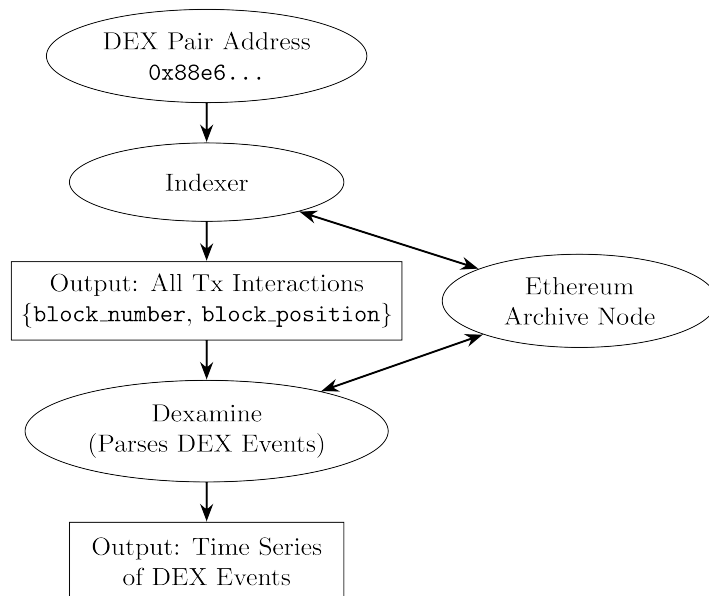


Figure 2: Dexamine workflow.

Dexamine example

Dexamine is designed to be user-friendly, adaptable, and efficient, making it an asset for anyone working with decentralized exchange data.

The following is a basic example of how to parse all market events (swaps, mints, burns) from a Uniswap V3 transaction:

```
# Import packages
from dexamine.parsers.uniswap_v3 import parse_uni_v3_events
from dexamine.shared import general_helpers
from dexamine.shared import constants

# Transaction hash
tx_hash = "0xf16f579a54c0d5700310ca54948d315b65b3f7d224a4af621ebfbaf29fcae8d5"
```

```

# Get logs from the transaction
receipt_data = general_helpers.get_receipt_data_by_hash(tx_hash)
logs = receipt_data['logs']

# Load ABIs
erc20_abi = general_helpers.load_abi(constants.path_erc20_abi)
uniswap_v3_pair_abi = general_helpers.load_abi(constants.path_uniswap_v3_pair_abi)

# Parse all events from Uniswap V3
events = parse_uni_v3_events.parse_all_v3_events(logs, erc20_abi, uniswap_v3_pair_abi)

```

Acknowledgements

I thank the teams at **Trueblocks** (TrueBlocks Team 2022) and **Erigon** (Erigon Team 2022) for their support on node operation and indexing, as well as community members of Flashbots and Uniswap for their help with understanding the DEX protocols.

References

- Adams, Hayden, Noah Zinsmeister, and Dan Robinson. 2020. “Uniswap V2 Core.” Tech. rep., Uniswap.
- Adams, Hayden, Noah Zinsmeister, Moody Salem, River Keefer, and Dan Robinson. 2021. “Uniswap V3 Core.” Tech. rep., Uniswap.
- Buterin, Vitalik. 2013. “A Next-Generation Smart Contract and Decentralized Application Platform.” White paper. <https://ethereum.org/en/whitepaper/>.
- Erigon Team. 2022. “Erigon.” <https://github.com/ledgerwatch/erigon>.
- TrueBlocks Team. 2022. “TrueBlocks: Lightweight indexing for any EVM-based blockchain.” <https://trueblocks.io/>.
- Wood, Gavin. 2014. “Ethereum: A Secure Decentralised Generalised Transaction Ledger.” Yellow paper. <http://gavwood.com/Paper.pdf>.